



100+ MLOPS SYSTEM DESIGN SCENARIOS

- ✓ Master production ML infrastructure .
- ✓ Clear Lead MLOps & AI System Design interviews .

MODEL PROD
SERVING & SLA

DRIFT & CI/CD
AUTOMATION

SCALE & COST
GPU INFRA

Swipe to explore MLOps scenarios



Design a **real-time feature store** for a credit card fraud detection system handling 50,000 TPS with sub-10ms SLA. Architect ingestion, storage, and retrieval layers.

High-throughput fraud detection requires dual-storage architecture to prevent training-serving skew and meet tight p99 latency SLAs.

Dual Storage Architecture:

- **Online Store (Low Latency):** Redis Cluster or Aerospike in-memory database to serve point-lookups under 5ms at 50,000 TPS.
- **Offline Store (Batch Training):** Delta Lake / Apache Iceberg on S3 for time-travel queries and point-in-time training sets.

Ingestion & Stream Processing:

- Use Kafka + Flink for real-time window aggregations (e.g., transaction count in last 5 mins).
- Dual-write pipeline ensures features simultaneously land in online Redis and offline S3.

Consistency & Skew Prevention:

- Enforce entity key schemas via Protobuf contracts.
- Run automated daily integrity checks between online Redis and offline Parquet snapshots.

Your model experiences severe **concept drift** in production after a UI redesign. Walk through your automated drift detection and retraining trigger pipeline.

Concept drift occurs when the statistical relationship between input features and target labels changes, degrading model accuracy without missing values.

Monitoring & Drift Detection Metrics:

- **Data Drift:** Calculate Kolmogorov–Smirnov (KS) test and PSI (Population Stability Index > 0.25) between training baseline and 24-hr inference logs.
- **Concept Drift:** Monitor performance metrics (F1 score, log loss) on delayed ground-truth feedback streams.

Automated Trigger Pipeline:

- When PSI threshold exceeds 0.2, trigger Airflow DAG to sample recent window data.
- Execute automated validation checks and initiate model fine-tuning.

Safety Guardrails:

New retrained model candidate must beat production baseline on a held-out benchmark set before advancing to shadow deployment.

What You'll Cover Inside



VOLUME 1

Feature Stores

Real-time Redis, Feast, Delta



VOLUME 2

Model Serving

vLLM, TensorRT, Triton



VOLUME 3

Drift & Monitor

PSI tests, Prometheus, Evidently



VOLUME 4

CI/CD/CT Pipeline

Kubeflow, MLflow, Airflow



VOLUME 5

Data Lineage

Time-travel, versioning, DVC



VOLUME 6

Cost Optimization

GPU batching, edge, fallback

You need to deploy a **70B parameter LLM** for low-latency inference. How do you design serving infra to minimize VRAM & GPU cost?

Serving 70B models requires FP8/AWQ quantization, tensor parallelism across GPUs, and KV-cache memory management.

1. Model Optimization & Quantization:

Quantize weights to AWQ/INT4 or FP8. A 70B model drops from 140GB VRAM (FP16) down to ~40GB VRAM, fitting onto two A10G (24GB) or one A100 (80GB) GPU.

2. High-Efficiency Engine (vLLM / TensorRT-LLM):

- **PagedAttention:** Eliminates KV cache memory fragmentation by allocating memory dynamically in physical blocks.
- **Continuous Batching:** Batches incoming queries at token level without waiting for full sequence completion.

3. Multi-GPU Tensor Parallelism:

Use PyTorch Tensor Parallelism across 2 GPUs connected via NVLink to keep per-token latency under 30ms.

A recommendation model trained offline performs poorly online. How do you architect a **shadow deployment & evaluation framework** to diagnose the gap?

Offline metric lift rarely translates directly to online conversion due to data leakage, serving latency, or sample selection bias.

Shadow Traffic Pattern:

Route 100% of production request payloads asynchronously to both Candidate Model B and Baseline Model A via an API gateway message queue (Kafka/EventBridge). Response from B is logged but not served to users.

Side-by-Side Online Telemetry:

- Log exact features served to Model B to verify no missing feature values in production.
- Calculate prediction divergence (Jensen-Shannon divergence) between A and B outputs in real-time.

Root Cause Isolation:

If JS divergence is high, check for feature pipeline latency (stale online features vs fresh offline training features).

Design a continuous training & deployment (CT/CD) pipeline that guarantees zero-downtime deployment and instant rollback upon anomaly detection.

Automated model CD must treat model artifacts like software binaries with strict canary release gates.

Pipeline Stages:

- **1. Automated Retraining:** Kubeflow pipeline triggers training upon data drift alert or schedule.
- **2. Model Verification:** Run automated evaluation against golden validation suite (accuracy, latency, bias).
- **3. Progressive Canary:** Route 5% → 20% → 50% → 100% traffic via Envoy/Istio service mesh.

Automated Rollback Circuit Breaker:

Prometheus monitors HTTP 5xx errors, p99 latency, and prediction distribution. If 5xx rate > 0.5% or latency > 200ms, Istio instantly flips traffic 100% back to previous stable model container tag.

Your data lake receives 100TB of streaming logs. Architect **data versioning & point-in-time correctness** for reproducible model training.

Without point-in-time join correctness, features computed from future events leak into past training labels (future leakage).

1. Storage & Table Format:

Store raw log streams in Apache Iceberg / Delta Lake format on S3. Iceberg maintains ACID transactions and immutable snapshot commits for historical time-travel queries.

2. Point-in-Time AS-OF Joins:

Feature stores (e.g. Feast/Tecton) perform AS-OF joins: matching observation event timestamp t with feature state at $t - \epsilon$, ensuring zero data leakage.

3. Metadata Lineage Tracking:

Bind model version `v1.4` to the exact Iceberg table commit hash `commit_id_89f1` in the MLflow tracking registry.

How do you architect a **multi-tenant model registry** for 500+ micro-models with strict RBAC, compliance auditing, and lineage tracking?

Managing hundreds of micro-models across enterprise business units requires centralized governance with decentralized execution.

Central Registry & Metadata Schema:

Use MLflow / Feast Registry backed by PostgreSQL. Each artifact entry stores model binary URI, Git commit hash, dataset URI, training metrics, and compliance approval status.

Role-Based Access Control (RBAC):

- Data Scientists: Read/Write draft models to `Staging` stage.
- Compliance / Lead Engineers: Exclusive permission to sign off and transition model tags to `Production`.

Auditing & Lineage Graph:

Store lineage DAGs in Neo4j/OpenLineage to trace every prediction back to its raw training dataset partitions and hyperparameter config.

Your real-time scoring service experiences **p99 latency spikes up to 3s during peak hours. Walk through how you isolate and eliminate inference bottlenecks.**

Latency spikes under high concurrency stem from CPU thread contention, blocking DB lookups, or un-batched GPU transfers.

Step 1: Distributed Tracing Diagnosis

Inject OpenTelemetry trace IDs across payload parsing, feature store lookup, TensorRT inference, and response serialization.

Step 2: Common Bottlenecks & Fixes

- **Synchronous Feature Lookups:** Switch feature store client calls from blocking I/O to async/gRPC pool.
- **GPU Memory Thrashing:** Enable Dynamic Batching in Triton Inference Server (`'max_queue_delay_microseconds = 5000'`).
- **Python GIL Lock:** Replace Unicorn/Python serving container with C++ Triton executor.

Result: Stabilized p99 latency under 45ms under 10,000 QPS load.

Design a cost-effective, **auto-scaling GPU inference cluster** on Kubernetes for multi-model serving with dynamic batching.

Provisioning dedicated GPU nodes per model leads to massive idle cost. Multi-model serving maximizes GPU utilization.

Multi-Model GPU Sharing (Triton / Ray):

Host multiple smaller models (BERT, XGBoost, ResNet) on a single shared GPU instance using NVIDIA MIG (Multi-Instance GPU) or shared Triton GPU memory contexts.

Custom Metrics Auto-Scaling (KEDA):

- Scale pods based on **GPU Queue Depth** and **Duty Cycle %** rather than CPU utilization.
- Scale node pool automatically using Karpenter with Spot GPU instances for 60% cost reduction.

Cold Start Optimization:

Pre-warm base Docker images containing CUDA drivers & model weight caches on node local NVMe SSDs.

You need to implement **distributed training across 128 GPUs** for a foundation model. Architect the parallelism & checkpointing strategy.

Models exceeding single GPU memory require hybrid parallelism techniques (3D Parallelism: Data, Tensor, Pipeline).

3D Parallelism Topology (DeepSpeed / Megatron):

- **Tensor Parallelism (TP=4):** Split layer weights across 4 GPUs within NVLink node.
- **Pipeline Parallelism (PP=4):** Distribute layers sequentially over InfiniBand interconnect.
- **Data Parallelism (DP=8):** Replicate combined model across 8 worker groups with ZeRO-3.

Fault-Tolerant Checkpointing:

Use async distributed checkpointing (TorchSnapshot) to write weights to high-speed NVMe scratch storage without stalling active training loops.

How do you design a centralized **prompt management & LLM evaluation platform** for 20 engineering teams to prevent prompt regressions?

Prompts are code. Without version control and automated evaluation pipelines, prompt tweaks cause silent downstream failures.

Centralized Prompt Registry:

Store system prompts as versioned templates in Git / LangSmith Registry with explicit variable schemas (e.g. `user\_query`, `context\_chunks`).

CI/CD Evaluation Gates:

- On every prompt PR, trigger automated test suite across 200 golden test cases using LLM-as-a-Judge.
- Evaluate metrics: RAG correctness, hallucination score, toxicity, and token output count.

Rollout Guardrails:

Block PR merge if accuracy drops > 1% or cost per query increases > 15% compared to main branch baseline.

Design an **active learning & annotation pipeline** that automatically identifies low-confidence predictions and routes them to human labelers.

Randomly labeling data is wasteful. Active learning selects only the most informative data points for human annotation.

1. Uncertainty Sampling Engine:

- Compute prediction entropy or margin sampling score on live inference streams.
- Filter samples where prediction confidence falls between 0.45 and 0.55.

2. Deduplication & Diversity Filtering:

Pass uncertain samples through an embedding clustering step (Faiss/K-Means) to drop near-duplicate instances before sending to labelers.

3. Human-in-the-Loop Feedback Loop:

Route filtered samples into Label Studio / Labelbox. Once annotated, automatically push approved labels into training data lake.

Your company requires GDPR compliance ("Right to be Forgotten") for ML models trained on user data. Architect a machine unlearning pipeline.

Completely retraining a large model from scratch every time a user requests data deletion is cost-prohibitive.

1. Data Sharding Strategy (SISA Architecture):

Partition training data into independent shards ($S_1, S_2, \dots, S_k$) and train isolated sub-models (slicing). When user data deletion occurs, only retrain the single affected shard.

2. Lineage Audit Purging:

- Purge user entity records from Feature Store online Redis and offline Parquet partitions.
- Maintain immutable deletion log proofs for regulatory compliance auditing.

3. Differential Unlearning:

For core models, execute differential gradient unlearning steps to scrub target sample influence without performance loss.

Architect an **edge AI deployment system** for 100,000 IoT devices with over-the-air (OTA) model updates and offline fallbacks.

Edge deployments operate under strict memory limits, intermittent connectivity, and constrained compute power.

1. Edge Model Compression:

Quantize FP32 models to INT8 using ONNX Runtime / TensorRT-Edge, reducing binary footprint by 75% with under 1% accuracy drop.

2. Differential OTA Updates:

- Deliver binary delta patches (using `bsdiff`) rather than full 500MB model re-downloads over cellular/IoT connections.
- A/B dual-partition flash storage on edge devices allows instant rollback if update verification fails.

3. Offline Fallback & Telemetry:

If device loses network connectivity, execute local quantized INT8 model; store telemetry logs locally in SQLite and sync once reconnected.

You have a **\$2,000/month infrastructure budget** to serve 5 million daily model predictions with 99.9% uptime. Architect this cheaply.

Step 1: Calculate unit economics

5M daily predictions $\times$ ~ 3 queries/day = 15M queries/month. $\$2,000 / 15M = \0.00013 per query budget limit.

Step 2: CPU-Only Inference Optimization

Export PyTorch models to ONNX Runtime with OpenVINO / AVX-512 optimizations. Run on CPU spot instances (c6i.xlarge) instead of GPUs (\$0.04/hr vs \$0.90/hr).

Step 3: Aggressive Response Caching

Deploy Redis cluster for prediction caching. 65%+ cache hit rate reduces required compute nodes from 12 instances down to 4.

Step 4: Final Fleet Cost

$\sim \$1,400/\text{mo}$ compute, \$200 Redis, \$150 storage, \$250 network = \$2,000/mo total.

100+ SCENARIOS LIKE THIS

Master Production MLOps & ML System Design.

WHAT WE COVERED:

- 15 Real-World MLOps Architecture Scenarios
- Feature Store & Serving Infra Math
- CI/CD/CT & Drift Detection Pipelines

Save & Repost for your next interview!
Follow Resnur AI for daily system design breakdowns.